

杂题选讲

kradcigam

江苏省常州高级中学

2026.5.16

使用说明

- 题目都有超链接，可以直接点开；
- 希望大家能认真思考！

旧事重提

题目描述

许多年前，有 n 只球队，两两之间有一场比赛，结果分为两种：

- 1 平局：双方各得 1 分；
- 2 决出胜负：胜者得 2 分，负者得 0 分。

小 O 只知道第 i 只球队最终的得分在 $[l_i, r_i]$ 之间。他还会有 m 次修改记忆，第 j 次将第 x_j 只球队的得分区间改为 $[p_j, q_j]$ 。

小 O 想知道至少需要修复多少次，才能使得存在可能的比赛结果，使第 i 只球队的总得分在 $[l_i, r_i]$ 之间，其中每次修复如下：

- 1 选择一个 i ，将 l_i 减小 1；
- 2 选择一个 i ，将 r_i 增加 1。

数据范围

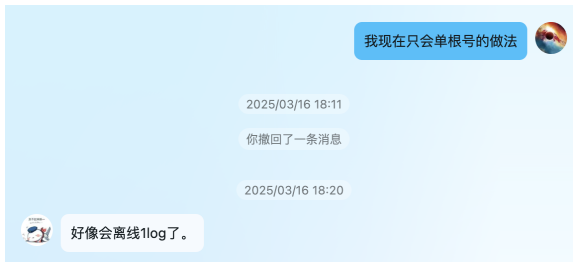
$$1 \leq n \leq 5 \times 10^5$$



闲话

很遗憾，本题无人通过。

最高分：ecnerwala（55 分，03:40:47），并有多位选手获得 30 分。



大家觉得这是谁在出题？

刻画充要条件

令第 i 个人的得分为 a_i ，我们考虑如何判断序列 a 是否可能为一组合法的得分方案。

结论

a 序列合法的充要条件为：

- $\sum_{i=1}^n a_i = n(n-1)$;
- 其次将 a_i 按照从小到大排序得到序列 a' ，需要满足 $\forall p \in [1, n], \sum_{i=1}^p a'_i \geq p(p-1)$ 。

证明

这两个条件显然是必要的。

证明

证明

考虑构建一张二分图，其中左边有 n 个点，表示 n 只球队，右边有 $\binom{n}{2}$ 个点，第 $(i, j) (i < j)$ 个点表示第 i 只球队和第 j 只球队之间的比赛。

从源点 S 向每个左部点 i 连 a_i 的流量，从每个右部点向汇点 T 连 2 的流量，并有 i 和 j 向 (i, j) 连边，流量为 $+\infty$ 。

于是就变成了二分图匹配模型，我们考虑 Hall 定理。于是二分图有完全匹配的充要条件为：对于左侧选择了点集 S ，有 $|N(S)| \geq |S|$ 即为：

$$2|S|(n - |S|) + |S|(|S| - 1) \geq \sum_{i \in S} a_i$$

证明

证明

这个形式并不好看，我们考虑其补集的形式。

我们考虑令 $T = \{1, 2, \dots, n\} \setminus S$ ，又由于两侧流量总和相同，则限制变为：

$$|T|(|T| - 1) \leq \sum_{i \in T} a_i$$

我们考虑最紧的条件，那么 T 一定为前 $|T|$ 小值，于是就得到了结论中的形式。

于是，我们对于 r 序列有限制：将 r 序列从小到大排序，前 i 个数的和至少为 $i(i-1)$ ；对于 l 序列，令 $l'_i = 2(n-1) - l_i$ ，则就变成了一样的形式。

发现关键性质

结论

对于限制序列 l 和限制序列 r ，只需要满足限制序列 l 、限制序列 r 分别合法，则限制序列 l 和限制序列 r 组合起来依旧合法。

证明

我们考虑序列 a ，初始有 $a_i = l_i$ ，每次我们找到 $a_i \neq r_i$ 中的最小的 a_i ，执行 $a_i \leftarrow a_i + 1$ 操作。我们会发现最终序列会形成：

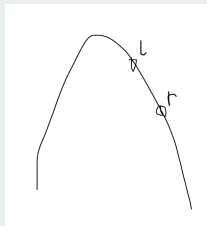
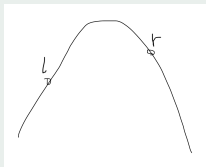
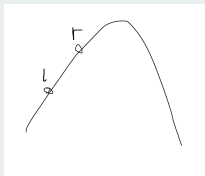
- 一段前缀为 r_i ；
- 中间一段分为两部分，前一部分为 p ，后一部分为 $p + 1$ ；
- 一段后缀为 l_i 。

我们考虑令 $b_i = a_i - 2(i - 1)$ ，则首先对于 r_i 部分，由于这些 r_i 一定是 r 序列中从大到小排好序的一段前缀；对于 l_i 部分同理。

证明

证明

考虑 $p, p, \dots, p, p+1, p+1, \dots, p+1$ 这部分。我们考虑对 $2(i-1)$ 做差，注意到这部分一定是一段前缀满足 $\geq 2(i-1)$ ，一段后缀满足 $< 2(i-1)$ ，所以对应到前缀和一定是一段单峰函数。由于单峰函数任取两个点，一定有一个点是区间最小值，所以中间这部分是符合条件的。



算法 1

我们考虑根号重构，对于第 i 个块的一个数。假设在其前面有 a 个数，和为 b ，且这个数在第 i 个块的第 c 个，前缀和为 d 。那么权值为 $(a+c)(a+c-1) - (b+d)$ ，容易转化成一次函数的形式。每次重构块时，对每个块维护凸包。注意到每次修改后，对于第 i 个块 c 只会最多变化 1。于是，我们建凸包时，要求如果一个点无法在任何整数取值中取到最值，我们也把它弹出去。这样一次修改，对于凸包上的最优值，只会至多移动一步。

时间复杂度 $O(n\sqrt{n})$ 。

算法 2

我们问题转化成了维护对序列 a 的单点修改，令 a 序列的前缀和序列为 $prea$ ，对固定序列 b ，令序列 b 的前缀和为 $preb_i$ ， $\max_{i=1}^n \{preb_i - prea_i\}$ 。

尝试删除 a_j 并分析其影响，令剩下的序列为 a' ，并令其前缀和为 $prea'$ ，则求 $\max_{i=1}^n \{preb_i - \min\{prea'_i, prea'_{i-1} + a_j\}\}$ ，也就是 $\max_{i=1}^{n-1} \{\max\{preb_i, preb_{i+1} - a_j\} - prea'_i\}$ 。

于是等价于对于序列 $preb$ ，执行 $preb'_i = \max\{preb_i, preb_{i+1} - a_j\}$ 的操作。

算法 2

考虑对 $preb$ 的差分也就是序列 b ，分析其影响。我们发现一定是一段后缀选择 $preb_{i+1} - b_i$ ，对应的前缀选择 $preb_i$ ，于是等价于在序列 b 中找到最后一个 $\leq a_j$ 的数 l 和第一个 $> a_j$ 的数 r ，并将 l, r 删除，加入 $l + r - a_j$ 。

同时，若不存在 l ，则我们对答案增加 $r - a_j$ ，即将 a_j 操作成 r ，不然就不满足条件。

我们考虑对修改进行线段树分治，同时在过程中维护序列，并使用双指针维护这个过程。

不难发现当前在线段树结点 x 后的序列长度，为修改区间经过这个线段树结点的次数。

时间复杂度 $O(n \log n)$ 。

Zeroing the segment

题目描述

给定一个长为 n 的正整数序列 $a_{1 \sim n}$ 。对于一个序列 $b_{1 \sim m}$ ：

- 初始有一常数 $x = 0$ ；
- 你可以进行若干次操作，操作共有两种：
 - 1 令 $x \leftarrow x + 1$ ；
 - 2 选择 $1 \leq i \leq m$ ，令 $b_i \leftarrow b_i \oplus x$ 。
- 定义序列 $b_{1 \sim m}$ 的权值为要使 $b_{1 \sim m}$ 均变为 0 所需的最小操作次数。

接下来有 q 次询问，每次询问给出 l, r ，求出 $a_{l \sim r}$ 的权值。强制在线。

数据范围

$$1 \leq n, q \leq 2 \times 10^5。$$



解法

首先 x 一定需要通过操作 1 操作到 $\max_{i=l}^r \{a_i\}$ 的二进制最高位 2^k 。

因此每个数至多需要操作两次。具体而言，对于数 a_i ：

- 若 $a_i \leq 2^k$ ，则需要 1 次操作 2；
- 若 $a_i > 2^k$ 时：
 - 若最终 $x \geq a_i$ 则需要 1 次操作 2；
 - 否则需要 2 次操作 2。

于是就变成了计算 $\min_{x \geq 2^k} \{x - \sum_{j=2^k+1}^x cnt_j\}$ 。
这其实等价于计算 $\max_{x \geq 0} \{\sum_{j=1}^x cnt_{2^k+j} - x\}$ 。

$$\max_{x \geq 0} \left\{ \sum_{j=1}^x cnt_j - x \right\}$$

我们发现如果我们把每个数 a_i 都看成一个二分图上的左部点 $t = a_i$ ，可以和二分图右侧的前 t 个点匹配。那么根据 Hall 定理， $\max_{x \geq 0} \left\{ \sum_{i=1}^x cnt_i - x \right\}$ 的实际含义是这张二分图最大匹配中，左部点失配点的数量。

于是，我们维护前 k 个点字典序最大的一组二分图匹配。

维护

动态维护匹配点集 S 。考虑每次新增一个 a_k ，那么相当于加入左部点 $t = a_k$ ，先判断这个点是否可以直接加入 S 中使得依旧存在一组完美匹配，如果不可以，则删掉当前 S 中原序列编号最小的一个，记其编号为 pos ，使得将其删除并将 t 加入 S ，存在一组完美匹配。

$$\max_{x \geq 0} \left\{ \sum_{j=1}^x cnt_j - x \right\}$$

具体维护时，我们考虑存在一组完美匹配，当且仅当记 $d_i = i - \sum_{j \in S} [j \leq i]$, $\forall i \in [1, n]$, $d_i \geq 0$ 。因此我们维护这样的数组 d ，如果加入点 t 时出现不合法，则当且仅当将 $[t, n]$ 区间 -1 ，出现了 -1 ，我们找到最左侧的 -1 位置 p 。那么我们就需要删除一个点 x 满足 $x \leq p$ ，并选择在原序列中编号最小的一个，这样可以使得 $[x, r-l]$ 区间 $+1$ ，此后便满足 $\forall i \in [1, r-l]$, $d_i \geq 0$ 的要求了。

因此，我们可以求出 pos_i 序列。当 $i \in [1, pos]$ 时，左部点失配点数量会增加 1，即 $\max_{t \in [0, r-l]} \left\{ \sum_{i=1}^t cnt_i - t \right\}$ 会增加 1。

解法

因此 $[l, r]$ 区间中：

$$\max_{x \geq 0} \left\{ \sum_{j=1}^x cnt_j - x \right\} = \sum_{i=l}^r [pos_i \geq l]$$

时间复杂度 $O(n \log n)$ 。

Increment and Swap

题目描述

有一个长度为 n 的数列 a 。

对于该数列，你可以进行以下两种操作：

- 交换相邻的两个元素。
- 任意选择一个元素，将它的值增加 1。

你可以多次执行上述操作，求将数列 a 变为单调不减序列所需的最少操作次数。

数据范围

$$1 \leq n \leq 2 \times 10^5, \quad 1 \leq a_i \leq 10^9。$$

结论

结论

答案为：

$$\sum_{i=1}^n \min_{j \geq a_i} \{j - a_i + \sum_{k=1}^{i-1} [a_k > j]\}$$

解法

必要性

首先不难发现，这个式子求出来的结果一定是答案的下界。这是因为对于 a_i 增加到 v 时， $a_1 \sim i-1$ 与 v 产生的逆序对数量至少是 $\sum_{j=1}^{i-1} \mathbb{1}_{[a_j > v]}$ 。但实际上 a_j 有可能会变大，所以产生的逆序对数量至少这么多。

证明续

充分性

接下来我们证明答案一定可以取到这个值。

令 $b_i = \operatorname{argmin}_{v \leq a_i} \{j - a_i + \sum_{k=1}^{i-1} \mathbb{1}_{[a_k > j]}\}$, 如有多个 v 能取到最小值, 取其中最小的 v 。

结论

$\forall i < j, b_j \notin [a_i, b_i)$ 。

证明

考虑反证。若 $b_j \in [a_i, b_i)$,

- 由 b_i 得: $\sum_{k=1}^{i-1} \mathbb{1}_{[b_j < a_k \leq b_i]} > 0$;
- 由 b_j 得: $\sum_{k=1}^{j-1} \mathbb{1}_{[b_j < a_k \leq b_i]} \leq 0$, 矛盾。



Potential Peak 改

题目描述

给定序列 $a_1, a_2, \dots, a_n$, 对所有前缀 i 独立求解下列问题。每次操作你可以选择:

- 1 单点修改: 选择一个 $j \in [1, i]$, $a_j \leftarrow a_j - 1$;
- 2 交换相邻位置: 选择一个 $j \in [1, i)$, $\text{swap}(a_j, a_{j+1})$ 。

要求 $a_1, a_2, \dots, a_i$ 单调不降, 求最少操作次数。

数据范围

$1 \leq n \leq 10^5$, $1 \leq a_i \leq n$ 。

结论

结论

求解前缀 k :

$$\sum_{i=1}^k \min_{v \leq a_i} \{a_i - v + \sum_{j=i+1}^k [v > a_j]\}$$

分析单调性

我们发现对于 i 而言，随着 k 的增加，最优的 v 是单调不降的。这里 $v \leq a_i$ 的限制非常难受，于是我们线段树分治，将 $[1, a_i]$ 这个值域划分成 $O(\log n)$ 个区间。那么，由于最优的 v 取值随着 k 增加单调不降，自然有对于每个区间，最优决策点 v 落在这个区间内，对应的 k 是一段区间。

接下来，我们先分析如果知道对于 i ， v 的取值在 $[l, r]$ 内，对于特定的 k 怎样能以一个更加易于拓展的形式来求解。

化简答案

记 cnt_t 表示 $\sum_{j=i+1}^k [a_j = r - t]$, 于是我们要求解的是:

$$\min_{v \in [l, r]} \left\{ a_i - v + \sum_{j=i+1}^k [v > a_j] \right\} \quad (1)$$

$$= \min_{v \in [l, r]} \left\{ k - i + a_i - v - \sum_{j=i+1}^k [v \leq a_j] \right\} \quad (2)$$

$$= k - i + a_i - \max_{v \in [l, r]} \left\{ v + \sum_{j=i+1}^k [v \leq a_j] \right\} \quad (3)$$

化简答案

$$=k-i+a_i+\sum_{j=i+1}^k[r\leq a_j]-\max_{v\in[l,r]}\{v+\sum_{j=1}^{r-v}cnt_j\} \quad (4)$$

$$=k-i+a_i+\sum_{j=i+1}^k[r\leq a_j]-\max_{t\in[0,r-l]}\{r-t+\sum_{j=1}^t cnt_j\} \quad (5)$$

$$=k-i+a_i+r+\sum_{j=i+1}^k[r\leq a_j]-\max_{t\in[0,r-l]}\{\sum_{j=1}^t cnt_j-t\} \quad (6)$$

$$\max_t \{ \sum_{j=1}^t cnt_j - t \}$$

我们考虑对值域上的每个线段树区间都先预处理出 pos 序列，并对每个点 i 维护出对于 k 的主席树，即维护出 $\sum_{j=i+1}^k [i \leq pos_j]$ 的主席树。

单次时间复杂度为 $O(n \log n)$ ，对每个值域上的线段树区间都求，时间复杂度 $O(n \log n \log a)$ 。

分界点

考虑如何求出点 i 的 $[1, a_i]$ 的 $O(\log a)$ 个值域线段树区间，最优决策点 v 落在这个区间内时，对应的 k 的区间。

由于决策是单调的，因此我们可以二分栈。于是现在问题转化成如何求出决策区间 $[l_1, r_1]$ 和 $[l_2, r_2]$ ($r_2 < l_1$) 之间 k 的分界点。

分界点

那么相当于是求出最小的 t 满足：

$$r_1 + \sum_{j=i+1}^t [r_1 \leq a_j] - \max_{t \in [0, r_1 - l_1]} \left\{ \sum_{j=1}^t cnt_{[l_1, r_1], j} - t \right\} \quad (7)$$

$$> r_2 + \sum_{j=i+1}^t [r_2 \leq a_j] - \max_{t \in [0, r_2 - l_2]} \left\{ \sum_{j=1}^t cnt_{[l_2, r_2], j} - t \right\} \quad (8)$$

$$r_1 + \sum_{j=i+1}^t [r_1 \leq a_j] - \sum_{j=i+1}^k [i \leq pos_{[l_1, r_1], j}] \quad (9)$$

$$> r_2 + \sum_{j=i+1}^t [r_2 \leq a_j] - \sum_{j=i+1}^k [i \leq pos_{[l_2, r_2], j}] \quad (10)$$

分界点

r_1, r_2 是常数, 另外 4 个量我们都可以用主席树维护, 因此可以在 4 个主席树上同时二分, 时间复杂度 $O(\log n)$ 。
由于有 $O(\log a)$ 个值域线段树区间, 时间复杂度 $O(n \log n \log a)$ 。

统计答案

最后我们考虑如何统计答案，我们对 k 从左到右扫描线，因此可以动态维护对于每个 i 的 $O(\log n)$ 次切换决策值域线段树区间的操作。

我们对每个值域线段树区间维护一个树状数组，维护当前在这个决策值域线段树区间内的 i 在值域上的前缀个数。当加入 k 时，我们找到 a_k 所在的 $O(\log a)$ 个决策值域线段树区间 $[l, r]$ 满足 $a_k \in [l, r)$ ，那么对于当前 i 在这个决策值域线段树区间内，且 $i \leq pos_{[l,r],k}$ 时， $\max_{t \in [0, r-l]} \{\sum_{j=1}^t cnt_j - t\}$ 会增加 1，那么答案就会减少 1。

时间复杂度

由于每次切换区间需要进行决策值域线段树区间上的树状数组单点加减，这部分的时间复杂度 $O(n \log n \log a)$ ，查询时需要在 $O(\log a)$ 个决策值域线段树区间查询树状数组值域前缀个数，时间复杂度也为 $O(n \log n \log a)$ 。

此外，统计

$k - i + a_i + r + \sum_{j=i+1}^k [r \leq a_j] - \max_{t \in [0, r-l]} \{ \sum_{j=1}^t cnt_j - t \}$ 部分的贡献是简单的。

因此，总时间复杂度为 $O(n \log n \log a)$ 。

比赛

题目描述

给定长度为 n 的环，把 $1 \sim n$ 这 n 个元素填入这个环，其中每个元素有红蓝之一的颜色。如果相邻两个元素异色，那么编号较大的元素所属的颜色得分 $+1$ 。

特别的，如果某个蓝色元素顺时针下一个元素是红色的，且红色元素权值较大，则这种情况不会让红色得分 $+1$ 。

对于 $k \in [-n, n]$ ，计数有多少个有圆排列，使得红色得分比蓝色得分多 k ，对 998244353 取模。

数据范围

$1 \leq n \leq 3000$ ，85 分： $n \leq 500$ 。

闲话

错完了，这场比赛做了比赛不仅这场比赛爆了，之后也不会正常打比赛了😞。

~~世界毁灭了，怎么 $O(n^4)$ 卡常是可以获得 85 分的。~~😞而我赛时简单尝试卡常无果之后就放弃了，写拆贡献的做法又一直在假。😞

解法

令环中 4 种相邻异色大小关系出现情况的次数分别为 $C_{B<R}$, $C_{B>R}$, $C_{R<B}$, $C_{R>B}$, 那么原题要求的就是 $C_{R>B} - C_{B>R} - C_{R<B}$, 看起来就非常奇怪且非常刻意, 这个不对称的东西实际上提醒我们需要拆贡献。

由于 RB 数量和 BR 数量一样, 所以

$$C_{B<R} + C_{B>R} = C_{R<B} + C_{R>B},$$

$C_{R>B} = C_{B<R} + C_{B>R} - C_{R<B}$, 因此原题要求的便是

$C_{B<R} - 2C_{R<B}$, 就只有 2 个量了, 并且还都是 $<$ 。

我们考虑容斥, 计算 $f_{x,y}$ 表示钦定有 x 个 $B < R$ 有 y 个 $R < B$ 的方案数, 然后再通过对两维分别二项式反演就能求出 $g_{x,y}$ 表示恰好有 x 个 $B < R$ 有 y 个 $R < B$ 的方案数, 进而计算出答案。

求解 f

由于 1 前的位置一定不会被钦定，考虑将 1 的位置放在开头，断环成链。我们发现钦定一组大小关系实际上相当于连一条边，这 $x + y$ 条边会形成 $n - x - y$ 条链。由于这是一张 RB 两种颜色的二分图， $R < B$ 是 R 的出边、 B 的入边； $B < R$ 是 B 的出边、 R 的入边，因此这两部分是独立的。

我们记连 x 条 $B < R$ 边的方案数为 p_x ，连 y 条 $R < B$ 边的方案数为 q_y 。两者都可以通过倒着 DP 求出。

那么， $f_{x,y} = p_x q_y (n - x - y - 1)!$ ，后面的阶乘是因为链之间的顺序可以随意排列。

时间复杂度 $O(n^3)$ ，二项式反演可以用 NTT 优化卷积做到 $O(n^2 \log n)$ 。

也许是更为高明的比赛策略

然后决定先做做 A，发现做连续段 DP 确实可以多项式复杂度，但感觉要记很多个元。考虑到做 CF2018F3 时太急着写记太多元的暴力导致简单做法没看出来的惨痛经历，我想觉得肯定有什么简单转化，还是别往记一堆元的暴力上面想了。因为 $n \leq 500$ 的包有足足 45 分，觉得目标应该是一个 $O(n^3)$ 做法。拆了会贡献之后貌似确实想出来一个，然后就试着写了一下，调着调着发现假了！此时已经过了 1h，我意识到我又犯错了：应该先写一个 $O(n!)$ 暴力确认贡献是否拆对的。感觉人有点红温，决定先换到 B 做一做。

B 感觉做法很显然啊！把这个过程画出来并倒着做，一下就看出来怎么做了！但一开始细节没编对，所以还是对拍调了一会，还剩 3h 时才过。

回过头来看 A，决定继续找简单转化。在试了很多很多种拆贡献的方法之后我终于发现了一个重要事实：按照顺时针转环，RB 的出现次数总和 BR 相等，而可以拆贡献使得这两种情形的权值分别是 1 和 -1 ！基于此又想了想终于得到了一个 $O(n^3)$ 做法，赶紧写了一下在还剩 1.5h 的时候拿到了 85 分。显然可以 FFT 优化到 $O(n^2 \log n)$ ，但最后要用 FFT 做的问题形式略有特殊，说不定会有更简单的做法从而标算不是这个。当时没有意识到虽然 $n = 3000$ 但其实常数可以分析出是比较小的，同时我只会背效率不太优秀的递归式 FFT 的板子，再考虑到最后一个包只有 15 分，决定还是先不写了，毕竟我 C 现在还是 0 分。

乘积的期望

题目描述

有一个长度为 n 的序列 $a_1, a_2, \dots, a_n$ 。初始序列的所有元素均为 0。再给定正整数 m 、 c 和 $(n - m + 1)$ 个正整数

$b_1, b_2, \dots, b_{n-m+1}$ 。

对序列 $a_1, a_2, \dots, a_n$ 进行 c 次操作，每次操作为：

- 随机选择整数 $1 \leq x \leq n - m + 1$ ，其中选到 y ($1 \leq y \leq n - m + 1$) 的概率为 $\frac{b_y}{\sum_{i=1}^{n-m+1} b_i}$ 。将

$a_x, a_{x+1}, \dots, a_{x+m-1}$ 增加 1。

c 次操作中对 x 的随机是独立的。求操作完成后序列中所有元素的乘积的期望，对 998244353 取模。

数据范围

$2 \leq m \leq n \leq 50$, $1 \leq c < 998244353$, $1 \leq b_i \leq 10^5$ 。



闲话

我不会分步转移😞。

半年后发现自己半年前就差个分步转移就足以 AC。

我的解法

这个题一看肯定是类似 Edge Subsets 要对 m, n 的倍数关系做分治。

算法 1: m 比较小的时候

这道题的一大难点在于: m 小的情况并不是很平凡, 但如果不会做这个情况可能这道题就拿不到什么分了。

对乘积拆组合意义, 于是就变成是每个位置对应了一次操作, 不同位置之间对应的操作可能是相同的。不过我们发现对于相同操作的不同位置, 我们只关心最小编号的位置和最大编号的位置。

正着 DP 可能不是很好做, 我们考虑倒过来 DP。记状态为 $f_{i,j,S}$ 表示当前从后往前在位置 i , 当前总共选择了 j 次不同的操作, i 往后的 m 个位置还没有确定其操作的集合为 S 。

转移

我们考虑加入位置 i 时的情况：

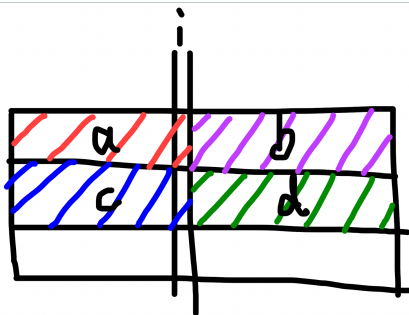
- 若 i 和 i 后面某个位置是同一个操作，则令最后的那个位置为 j ，那么 j 操作的开头一定 $\leq i$ ，且 j 一定在集合 S 中，因此转移为 $f_{i,j,S} \leftarrow |S|f_{i+1,j,S}$ ；
- 若 i 不和后面某个位置是同一个操作，转移为 $f_{i,j,S \cup \{i\}} \leftarrow f_{i+1,j,S}$ 。

因此此时集合 S 中一定是存着若干本质不同的操作。然后再考虑 i 的操作能和 S 中操作匹配的情况，枚举 S 中的编号并讨论是否匹配即可。

时间复杂度 $O(nm^22^m)$ 。

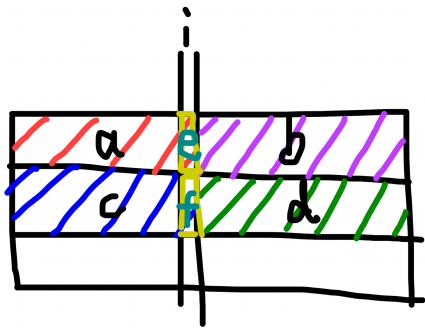
算法 2: m 比较大的时候

首先不难发现答案一定是关于 c 的不超过 n 次多项式, 因此我们只需要对 $c = 1, 2, \dots, n$ 都求出答案, 然后拉格朗日插值即可求出答案。当 $m > 16$ 时, 则由于 $n \leq 50$, 那么 $\frac{n}{m} < 3$ 。因此我们可以看成是一个 $3 \times m$ 不满的网格, 每次进行操作。考虑一个非常暴力的做法: 记 $f_{i,a,b,c,d}$ 表示:



转移

枚举第 1 行，第 i 列操作了 e 次；第 2 行，第 i 列操作了 f 次即可完成转移。



时间复杂度 $O(n^7)$ ，无法通过。但对于 $c \leq n$ 的部分分，状态可以少一维，此时时间复杂度变为 $O(n^6)$ ，足以获得 90 分。

优化

很难发现 e 和 f 的转移是完全独立的，分开转移即可。
时间复杂度 $O(n^6)$ 。

能量分配

题目描述

有 n 颗不同的宝石和 m 位学生，对于所有宝石分配给学生的方案，计算权值和，对 317 取模：

记第 i 位学生拿到的宝石数量为 c_i ，满足 $\sum_{i=1}^m c_i = n$ 。第 i 位同学的满意度 a_i 初始为 1，考虑其他所有同学 $j \neq i$ ：

- 若 $c_j > c_i$ 则 $a_i \leftarrow a_i \times A_{i,j}$ ；
- 若 $c_j = c_i$ 则 $a_i \leftarrow a_i \times B_{i,j}$ 。

一个方案的权值为 $\prod_{i=1}^m a_i$ 。

有 q 次询问，每次给定 n 。

数据范围

$1 \leq q \leq 100$, $1 \leq n \leq 10^{18}$, $1 \leq m \leq 12$ 。

闲话

这个题确实还是太变态了。
有人赛时这道题做了 2.5h 没调出来，宇宙级战犯。
是谁呢？😞

小模数的性质

以下记 $P = 317$ 。

首先对于固定的 c_i 序列，其方案数为 $\frac{n!}{\prod_{i=1}^m c_i!}$ 。

Kummer 定理

p 在 $\binom{n+m}{n}$ 中的幂次等于等于 $n + m$ 在 p 进制表示下进位的次数。

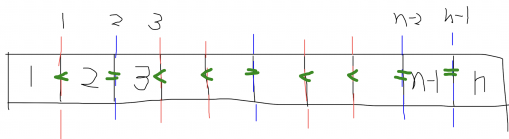
因此 $\frac{n!}{\prod c_i!} \not\equiv 0 \pmod{P}$ 当且仅当 $\sum c_i$ 做加法时没有产生进位。而此时，方案数在在 P 进制下每一位上是独立的。

解法

核心观察

考虑如何处理 A, B 权值的计算。首先我们发现，我们可以在最终排名（所有无标号同学之间的相对大小关系）确定下来后，再去关心学生和排名位置的对应关系，此时只需要进行形如将“人”填进排名的过程。

而“所有无标号同学之间的相对大小关系”可以用 $n-1$ 个二进制位来表示：



解法

所以，我们的计算过程可以分成两个部分：

- 1 对于每个最终“所有无标号同学之间的相对大小关系”，计算对应的 A, B 部分权值和，即“将人填入排名的过程”；
- 2 对于每次询问的 n ，求对应每个“所有无标号同学之间的相对大小关系”的方案数。

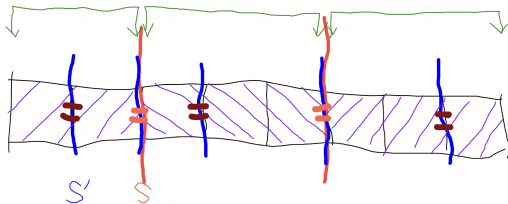
第一部分

这部分对时间复杂度的要求并不高，我们考虑一个暴力做法。
记 f_{S_1, S_2} 表示当前已经确定了选手集合 S_1 ，以及他们对应的“无标号相对大小关系”。转移时枚举 S'_1 满足 $S'_1 \cap S_1 = \emptyset$ ，即枚举“无标号相对大小关系”中在 S_1 前的一段相等的集合，并预处理 S_1, S'_1 之间的相对贡献。

时间复杂度 $O(\sum_{i=1}^m \binom{m}{i} 2^i 2^{m-i}) = O(2^m \sum_{i=1}^m \binom{m}{i}) = O(4^m)$ ，已经足够优秀。

第二部分

我们考虑先将 n 拆成 P 进制，假设有 k 位，第 i 位为 p_i 。
记状态为 $f_{i,S}$ 表示考虑完前 i 位 P 进制位，“所有无标号同学之间的相对大小关系”为 S 的方案数。考虑 $f_{i,S} \rightarrow f_{i,S'}$ 的转移 ($S \subseteq S'$)，对这部分预处理转移系数 g_{S,S',p_i} 。
考虑如何计算 $g_{S,S',i}$ ，我们发现这个部分等价于对于 S 内的每个连续段先分别预处理 $h_{S,j}$ ，再对 i 这一维做加法卷积。



计算 h

我们考虑一边搜索，一边记录 DP 数组 $dp_{i,j}$ 表示上一个数的值为 i ，总和为 j 的方案数。

根据 S 下一位的取值进行转移。

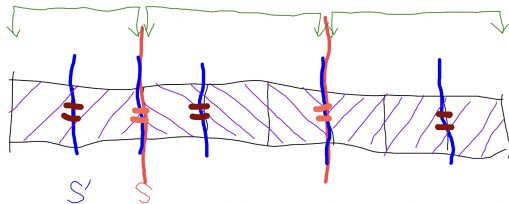
时间复杂度 $O(2^m P^2)$ ，已经足够通过。

一个优化是，我们可以从大往小搜索，这样搜到第 x 个数时， $j \geq ix$ ，所以 $i \leq \frac{n}{x}$ 。

时间复杂度 $O(2^m P \log P)$ 。

计算 f

直接暴力枚举。时间复杂度 $O(3^m m P^2)$ ，无法通过。
但我们发现



绿色部分的顺序是不重要的，对这一维加入记忆化就足以通过。

fun fact

```
sort(all(g));
if (mp.count(g)) {
    auto gw = mp[g];
    F(t, 0, MOD - 1) w[tot][t] = gw[t];
} else {
    array <int, MOD> t1;
    t1.fill(0);
    t1[0] = 1;
    for (int t: g) {
        F(a, 0, MOD - 1) {
            int g = ww[t][a];
            if (g) {
                F(b, 0, MOD - 1 - a)
                    t2[a + b] += C[a + b][a] * t1[b] * g;
            }
        }
        F(a, 0, MOD - 1) {
            t1[a] = t2[a] % MOD;
            t2[a] = 0;
        }
    }
    mp[g] = t1;
    F(a, 0, MOD - 1) {
        w[tot][a] = t1[a];
        t1[a] = 0;
    }
}
```

这样就足以通过！

这么复杂的题目，对我这种老年退役菜鸡还是太不友好了!!! 😞

Arkady and Rectangles

题目描述

按顺序在二维平面上画 n 个颜色为 $1, 2, \dots, n$ 的矩形（数字大的颜色覆盖数字小的颜色），问最后能看到多少种颜色。

数据范围

$$1 \leq n \leq 10^5。$$

闲话

这个题做法挺多的，讲讲我的做法。

解法

我们对 X 轴扫描线，对 Y 轴维护线段树。

对于线段树的每个结点，维护一个 set 表示当前覆盖这个区间中的所有颜色。

记 tag_i 表示第 i 号结点距离上次 pushdown 的所有时刻中，覆盖这个区间的值中最大值，最小是多少。

对于一个矩形插入、删除，我们对对应的 set 进行修改即可。

我们发现 tag 发生变化当且仅当一个矩形被删除后、一个矩形插入前的对应线段树结点，于是递归修改即可。

时间复杂度 $O(n \log^2 n)$ 。

Minimums or Medians

题目描述

你有一个包含 $1 \sim 2n$ 共 $2n$ 个整数的集合 S 。你必须执行恰好 k 次操作，每个操作都是以下两种其中之一：

- 将 S 中第 $1, 2$ 个元素删去。
- 将 S 中第 $\frac{|S|}{2}, \frac{|S|}{2} + 1$ 个元素删去。（显然 $|S|$ 一直是偶数，所以 $\frac{|S|}{2}$ 也一定是整数）

请你统计，通过这些操作可以获得多少个本质不同的最终集合 S ？答案对 998244353 取模。

数据范围

$$1 \leq k \leq n \leq 10^6。$$

解法

我们考虑找充要条件：

- 1 注意到所有被删除的连续段长度都是偶数。并且不同的连续段之间，都是被分开删除的。
注意到只有从 1 开始的连续段才可能用操作 1 删除，于是其它被删的数都是通过操作 2 删除的。
- 2 注意到第 i 次若为操作 2，则删除的较大数为 $i + n$ ，所以，被删除的数都必须 $\leq n + k$ 。
- 3 注意到若删除了数 $t \leq n$ 不在第一段，则 $[t, 2n - t + 1]$ 的数都必须被删除。

解法

上述 3 个必要条件是充分的，因为考虑我可以通过构造，每次能用操作 2 就用操作 2，否则就用操作 1，不难发现一定是合法的。设函数 $f(n, m)$ 表示长度为 n 的段中，选出 $2m$ 个数，并要求选出的数形成长度为偶数的连续段。考虑捆绑法，我们没选出一个数就要求必须选它后面的那个数，于是这样就只有 $n - m$ 个物品，从中选出 m 个，所以是 $\binom{n-m}{m}$ 。

解法

考虑如何计数。

首先，我们枚举从 1 开始的连续段需要操作的次数 t ，因为涉及到条件 3，所以我们需要根据是否覆盖超过一半来分类讨论：

- 1 $2t < n$ ，于是再枚举在 n 之前被删掉的连续段长度 c ，于是得到：

$$\sum_{t=1}^{\min\{k, \lfloor \frac{n-1}{2} \rfloor\}} \sum_{c=0}^{\min\{k-len, n-2k-1\}} f(k-c, k-t-c)$$

注意到 len 增加时，组合数的变化可以 $O(1)$ 维护，类似莫队移动指针，加入或删除组合数。

Alice, Bob, and two arrays.

题目描述

给定两个数组 a, b 和一个初始为空的数组 c 。Alice 和 Bob 轮流在 c 的末尾加入一个数，要求 c 始终是 a 和 b 的公共子序列，无法操作者失败，Alice 先手。

共有 q 场独立游戏。每场游戏会先删除 a 的前 x_i 个元素和 b 的前 y_i 个元素，并保证删除后两个数组的剩余部分以相同的值开头。你需要判断在双方最优策略下，Alice 是否获胜。

由于数组长度很大，数组 a, b 以若干个连续相同元素组成的段来表示。

数据范围

$$1 \leq N, M \leq 10^9, 1 \leq n, m, k \leq 1600, 1 \leq q \leq 10^6。$$

解法

首先有一个暴力做法，记 $f_{i,j}$ 表示当前 A 序列在 i ， B 序列在 j ，最后谁输谁赢，其中 i, j 颜色相同。

首先如果先手能转移到一种不同的颜色，满足先手必败。那么先手就必胜了，这个是好维护的。

那么剩下的情况是，不停在同一个颜色上跳，然后直到出现能跳到另一种颜色并获得胜利，或者以这种颜色结束。我们考虑定义值 $S(i, j)$ ，表示第一个序列前 i 个位置中，这种颜色的出现次数-第二个序列前 j 个位置中，这种颜色的出现次数。那么，首先这么跳的过程中， $S(i, j)$ 是不变的。能跳到另一种颜色 win，或者这种颜色结束了，可以看成是两种终止状态。

解法

于是整个过程就形如：我们记当前我们在第一个序列的第 i 个位置，在第二个序列的第 j 个位置，这种颜色在第一个序列前 i 个位置中出现了 a 次，在第二个序列的前 j 个位置中出现了 b 次，于是转移就是右上的一条射线。每次移动， a 会增加 1， b 也会增加 1，而一种终止情况，对应了一个矩形，碰到这个矩形就结束了。

所以等价于 $a - b$ 是一个区间。那么，我按照 i 从大到小， j 从大到小，每次做区间覆盖，就做完了。

时间复杂度 $O(nm \log n + q \log n)$ 。

谢谢大家

Thanks!

祝大家在 NOI2026 中取得好成绩!